

Zafaro Federico, mat. 337690, febbraio 2026

Università degli Studi di Trieste
Corso di Laurea Magistrale in Fisica

LABORATORIO DI FISICA COMPUTAZIONALE
PROGETTO FINALE
TRAVELING SALESMAN PROBLEM

Indice

1	Introduzione	1
1.1	Formalizzazione del problema	2
1.2	Implementazione degli operatori	4
1.3	Il codice	5
1.3.1	Setup	7
1.3.2	Sequenza di Metropolis	7
2	Misure preliminari	8
2.1	Nota sulla termalizzazione	8
2.2	Ricerca della temperatura critica	8
2.3	Protocolli di raffreddamento	11
2.4	Ottimizzazione del protocollo gaussiano	13
3	Calcolo e risultati	15
4	Conclusione	17

1 Introduzione

Si considerino N città entro una regione geografica non meglio specificata e un venditore ambulante che deve presentarsi in ciascuna di esse. In quale sequenza deve visitarle per seguire il percorso più breve?

Questo è il quesito centrale del *traveling salesman problem* (TSP). Lo studio di questo problema è di interesse per diversi ambiti, sia accademici sia industriali. Basti pensare ad una compagnia di trasporti e logistica: è nel loro interesse costruire un navigatore che consenta all'autista di finire le consegne nel minor tempo possibile, risparmiando carburante e aumentando la produttività. D'altra parte, non è sempre chiaro come collegare condensatori, resistenze, ponti e altri componenti elettronici in un circuito stampato, e non è banale minimizzare la lunghezza totale attraverso cui il segnale deve propagare, o il modo più efficiente di disporre i cavi (in parallelo, attraverso zone particolari, *et cetera*).

L'impostazione parte da pochissimi assunti essenziali e da alcune condizioni, che spesso si possono rilassare. Anzitutto, sono noti N indici che indicano N siti distinti su un reticolo non meglio definito, grazie al quale è possibile definire una distanza reciproca che li separa. Si richiede di costruire un percorso chiuso il più breve possibile che, come sopra, li colleghi tutti una sola volta. Si può subito richiedere, ad esempio, che un certo sito venga visitato due volte: questo però equivale a inserire un nuovo indice, legato ad un sito le cui coordinate sono esattamente quelle del sito che va visitato due volte, e richiedere che le due visite non avvengano consecutivamente. Inoltre, di solito, si considerano problemi simmetrici, dove andare dal sito i al sito j è come andare da j a i . Ma anche questa è una condizione che si può rilassare: la distanza tra due siti può essere diversa a seconda del verso di percorrenza, oppure subire un *bonus* (qualora sia una tratta "favorita": un'autostrada, un tratto di fibra ottica) o un *malus* (l'attraversamento di un fiume), o ancora può dipendere dal tempo.

La flessibilità del problema non lo rende però di facile applicazione. Calcolare analiticamente il percorso ottimale richiede un tempo di calcolo che scala molto più rapidamente di qualsiasi polinomio, perché N siti implicano l'esistenza di $(N - 1)!$ percorsi che li collegano. Perciò, il *traveling salesman problem* appartiene alla categoria dei cosiddetti *NP-hard problems*. Alcuni esempi di metodi esatti per il calcolo della lunghezza minima sono il *BRANCH & BOUND*, che esplora tutti i cambiamenti possibili e "pota" le ramificazioni peggiorative, e il *CUTTING PLANES*.

Nondimeno è ragionevole supporre di poter raggiungere un minimo locale della lunghezza possibile del percorso, che si avvicini quanto basta al minimo assoluto. Gli sforzi della comunità scientifica interessata all'argomento si concentrano dunque sul trovare metodi sempre più efficaci e rapidi di stimare un percorso sufficientemente buono, tramite l'implementazione di diversi metodi non-esatti.

- **SIMULATED ANNEALING** - Le varie configurazioni del sistema, cioè i vari percorsi che si possono formare tra i siti, occupano ciascuno un sito in una sorta di "spazio delle fasi" discretizzato. Si cercano alcuni protocolli, o "operatori", che agiscano sulla

configurazione corrente proponendo delle modifiche. Queste possono essere accettate o rifiutate probabilisticamente. Il peso probabilistico viene fatto dipendere da una temperatura fittizia: il sistema viene fatto "raffreddare" lentamente, in maniera che possa esplorare liberamente l'*energy landscape* a disposizione e assestarsi in un minimo sufficientemente profondo.

- GENETIC ALGORITHMS - Sul modello dell'evoluzione biologica, un *pool* di "cromosomi" (percorsi "genitori" creati casualmente) viene fatto accoppiare a generare nuovi percorsi, che popoleranno il *pool* successivo. Le strategie di accoppiamento, intese come i protocolli di creazione dei percorsi "figlio", sono al centro dell'ottimizzazione. Il processo viene ripetuto per diverse iterazioni, dette "generazioni", e l'obiettivo è ridurre al massimo la lunghezza del percorso entro il minimo numero di generazioni.
- ANT COLONY - Il problema viene mappato in un sistema collaborativo, come una colonia di formiche.
- PARTICLE SWARM - Il problema viene mappato in uno "sciame" di configurazioni diverse che collaborano tra di loro.
- NEURAL NETWORKS
- TABU SEARCH

Il presente progetto si occupa di offrire un ulteriore tentativo all'implementazione ottimizzata di un metodo di *simulated annealing*, confrontandolo poi con risultati ottenuti da alcuni algoritmi genetici.

1.1 Formalizzazione del problema

L'idea fondamentale di ogni iterazione è identificare un nuovo percorso, auspicabilmente più breve del precedente, e calcolarne la lunghezza totale. Come prima cosa, quindi, è essenziale poter individuare le distanze reciproche tra i siti.

Scelto un problema da analizzare, vale a dire un reticolo su cui sono posizionati i siti, la scelta convenzionale è di popolare una matrice dove l'entrata ij -esima corrisponde alla distanza tra il sito i e il sito j . Questa matrice ha sempre diagonale nulla, ovviamente, ed è simmetrica se il problema è simmetrico, antisimmetrica altrimenti. Dentro tale *distance matrix* è contenuta tutta l'informazione preliminare sul sistema: le distanze, eventuali scelte preferenziali per determinati percorsi ed eventuali asimmetrie. Gli indici dei vari siti e le loro coordinate completano il pacchetto che definisce univocamente un problema.

In secondo luogo, l'evoluzione del percorso avviene tramite protocolli, chiamati spesso "operatori" in letteratura, che agiscono sull'ordine di percorrenza, proponendo dei cambiamenti alla *schedule* del percorso. Quattro sono i principali, tra cui quelli proposti in [1], [2] e [3]:

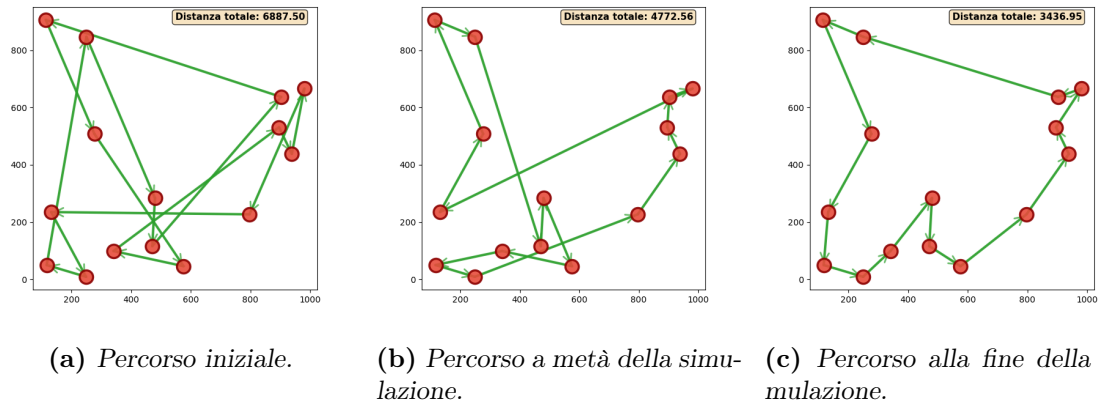


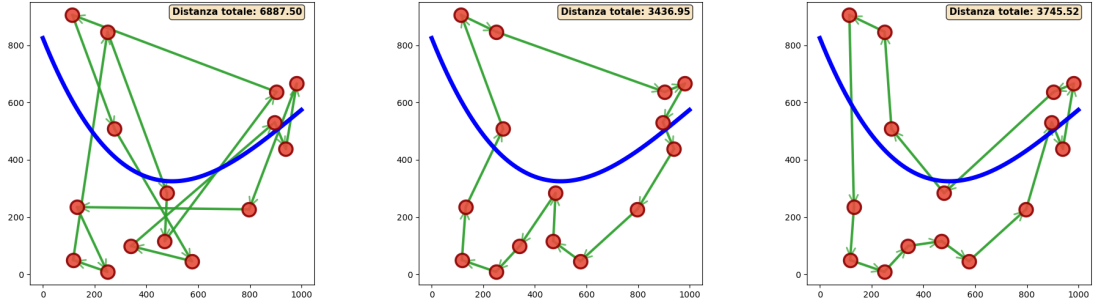
Figura 1.1 – Esempio di evoluzione di un problema simmetrico tramite *Simulated Annealing*, dove il verso di percorrenza non cambia le distanze. All'inizio il percorso è lungo e articolato, ma lentamente il sistema raggiunge uno stato di "minima energia", da intendersi come una soluzione quasi-ottimale.

- *swap* - due città casuali vengono scambiate di posto.
- *transport* - una sequenza di città viene estratta dal percorso e riposizionata altrove.
- *reverse* - una sequenza di città viene invertita.
- *2-opt* ("*two-options*") - un "nodo" formato da quattro città viene sciolto.

Le caratteristiche di ciascun protocollo verranno approfondite nel paragrafo 1.2. Rimane da formalizzare un protocollo di raffreddamento: un liquido diventa liquido sottoraffreddato quando a contatto con un termostato che si raffredda lentamente, ma in questo caso la temperatura è fittizia e quindi non si hanno indicazioni su quale andamento seguire. In generale, lungo una simulazione di n passi Monte Carlo, la temperatura sarà un array unidimensionale di n numeri decrescenti. La ricerca del protocollo di raffreddamento ottimale viene affrontata in dettaglio al paragrafo 2.3.

Va segnalato fin da subito che, dei quattro protocolli proposti, due sono molto grossolani (*swap*, *transport*) e uno molto fine (*2-opt*), quindi i primi hanno senso di esistere soprattutto ad alte temperature, quando il sistema sta ancora esplorando l'*energy landscape*, mentre l'altro è inutile se non a basse temperature, quando è necessario un lavoro più fine. Per questo motivo, i protocolli verranno fatti dipendere dalla temperatura, a seconda della preferenza che si vuole imporre.

In questo modo, si può implementare un algoritmo Monte Carlo che, lungo n iterazioni, scelga un operatore, lo applichi al percorso e valuti se accettare il cambiamento tramite algoritmo di Metropolis.



(a) Percorso iniziale.

(b) Risultato se attraversare il fiume è penalizzante.

(c) Risultato se attraversare il fiume è favorito.

Figura 1.2 – Esempio di evoluzione di un problema simmetrico con un termine di perturbazione: la regione è attraversata da un fiume, identificato dalla linea blu. Il sistema minimizza gli attraversamenti se attraversare il fiume è penalizzante, mentre li massimizza se è favorito. In questo caso, la matrice delle distanze è stata modificata per aggiungere un termine J alle coppie che stanno su sponde opposte del fiume, dove $J > 0$ indica una penalizzazione e $J < 0$ un vantaggio.

1.2 Implementazione degli operatori

Siano dati la matrice delle distanze, il vettore degli indici delle città che indica l'ordine di percorrenza (*path*) e le coordinate dei siti. D'ora in poi T rappresenterà il vettore delle temperature.

Swap. Due indici casuali nel *path* vengono scambiati. Gli indici non devono essere adiacenti: questo risulterebbe in un protocollo *reverse* applicato ad un segmento di due siti. Questo operatore rappresenta un'azione grossolana, da svolgersi preferibilmente ad alte temperature, e viene fatto dipendere linearmente da T .

Transport. Presi due indici casuali in ordine crescente, le città ad essi corrispondenti e quelle comprese vengono reinserite nel nuovo *path* in una posizione diversa. Anche questo è un protocollo grossolano e dipende linearmente da T .

Reverse. Presi due indici casuali in ordine crescente, le città ad essi corrispondenti e quelle comprese vengono reinserite nel nuovo *path* in ordine inverso. Questa azione è meno invasiva delle precedenti e ha buona applicabilità a ogni temperatura: a questo operatore viene associata una probabilità costante.

2-opt. Questo operatore viene attivato solo se viene rilevata un'intersezione nel percorso: quattro città, anche non adiacenti nel percorso, vengono visitate formando due segmenti secanti. Verificata la condizione e scelto questo operatore, l'intersezione viene sciolta. In un qualsiasi triangolo, la somma di due lati è maggiore che la lunghezza del terzo; pertanto,

visitare le quattro città senza intersezione è sempre una scelta favorevole. Questo operatore è preferibile a basse temperature, perché i protocolli più grossolani tendono a creare nodi, quindi dipende dall'inverso di T .

Per ogni passo dell'algoritmo, l'operatore da applicare viene scelto tramite trasformata inversa. Le probabilità sono:

$$p(\text{swap}) = T/N \quad , \quad p(\text{transport}) = T/N \quad , \quad p(\text{reverse}) = 1/N \quad , \quad p(2\text{-opt}) = 1/TN \quad (1.2.1)$$

dove N è il fattore di normalizzazione:

$$N = \begin{cases} \text{con intersezioni: } 2T + 1 + 1/T \\ \text{senza intersezioni: } 2T + 1 \end{cases} \quad (1.2.2)$$

Sia ora P la probabilità cumulativa discreta data dalle probabilità degli operatori. Se 1 = swap, 2 = transport, 3 = reverse e 4 = 2-opt, allora:

$$P(1) = T/N \quad , \quad P(2) = 2T/N \quad , \quad P(3) = (2T + 1)/N \quad , \quad P(4) = (2T + 1 + 1/T)/N \quad (1.2.3)$$

Generato un numero casuale, l'algoritmo sceglie l'operatore applicando P^{-1} al risultato.

L'operatore propone a sua volta una modifica del *path*, che viene accettata o rifiutata tramite l'algoritmo di Metropolis, usando come probabilità un equivalente del peso di Boltzmann, con la lunghezza del percorso al posto dell'energia e T come temperatura:

$$P(\pi) = e^{\ell(\pi)/T} \quad (1.2.4)$$

dove π è un dato *path* e $\ell(\pi)$ è la lunghezza del percorso identificato da esso.

Nello specifico, viene scelto un numero casuale $r \in (0, 1)$ e la modifica $\pi \rightarrow \pi'$ viene accolta se:

$$r \leq \min \left\{ 1, \frac{\ell(\pi')}{\ell(\pi)} \right\} \quad (1.2.5)$$

Bisogna osservare che, in particolare, l'operatore *2-opt* propone sempre una modifica positiva, per costruzione, e pertanto produce sempre $\ell(\pi') > \ell(\pi)$. Pertanto, il tasso di accettazione di questo operatore è 100%.

1.3 Il codice

Il codice utilizzato nello specifico interfaccia un Jupyter Notebook su Python (versione 3.13) con un file Fortran90, tramite `f2py-jit`. L'analisi dati viene fatta su Python mentre

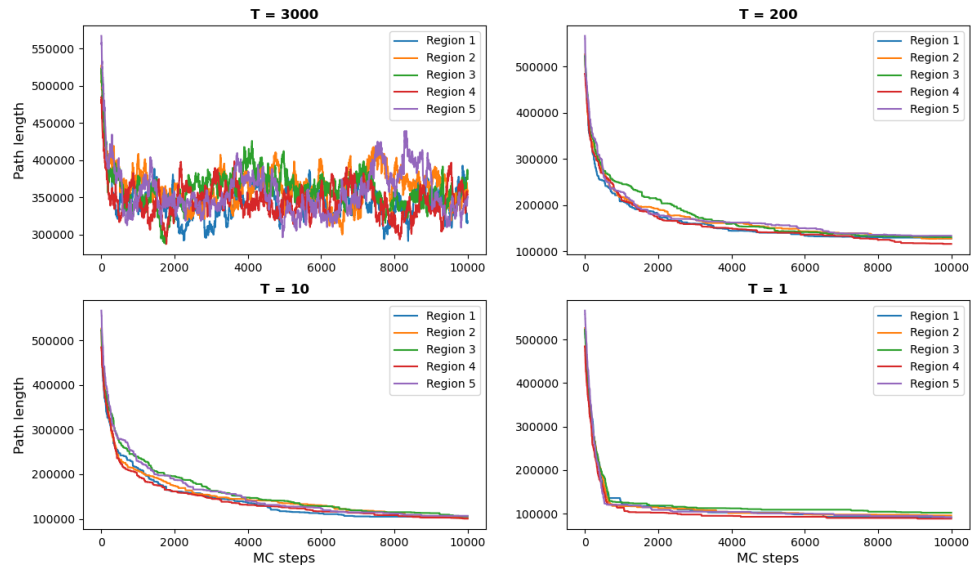


Figura 1.3 – Prove "esplorative" di scan in temperatura: un sistema di 100 città su un reticolo quadrato di lato 10^4 viene generato e fatto evolvere con 5 seed diversi per 10^5 step Monte Carlo. La temperatura viene mantenuta costante e assume quattro valori diversi nei quattro grafici. Si osserva come un primo cambiamento è visibile quando la temperatura passa da $\sim 10^3$ a $\sim 10^2$, e poi da 10^1 a 10^0 , mentre le finestre centrali non sembra mostrare cambiamenti.

il lavoro di calcolo e simulazione è principalmente assegnato a Fortran. Di seguito un elenco delle *subroutines* utilizzate.

1.3.1 Setup

Subroutines per generare un sistema su cui lavorare o per importarne uno dalla libreria del *traveling salesman problem*. Le seguenti sono presenti in duplice copia, per accettare input di coordinate intere e reali.

`read_tsplib_coords` - Importa e legge le informazioni di setup dei file in TSPLIB.

`create_city_array` - Genera un reticolo quadrato e colloca un dato numero di città su di esso, disponendole casualmente.

`get_travel_distance` - Dato un *path* e le coordinate delle città che esso attraversa, calcola la lunghezza totale del percorso.

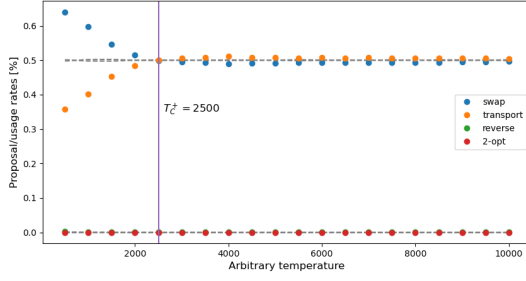
1.3.2 Sequenza di Metropolis

`choose_protocol` - Sceglie l'operatore in base alla temperatura, applicando la trasformata inversa.

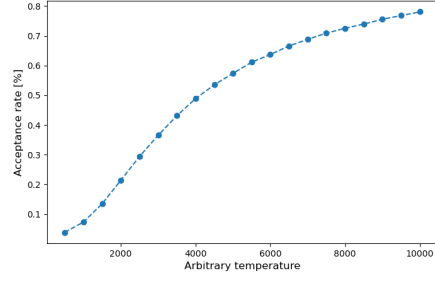
`swap`, `transport`, `reverse`, `two_opt` - Applicano l'operatore corrispondente.

`metropolis_step` - Svolge un passo di Metropolis, confrontando le probabilità, accettando o meno la modifica, e aggiornando le quantità interessate.

`metropolis_run` - Ripete `metropolis_step` per il numero di passi richiesto.



(a) Rates degli operatori proposti e usati.



(b) Percentuale di accettazione.

Figura 2.1 – Scansione in temperatura grossolana. Le linee tratteggiate grigie rappresentano i rates di proposta dei vari operatori, che rimangono a 0 per *reverse* e *2-opt* e a 0.5 per *swap* e *transport*. I dati colorati sono invece i rates di utilizzo corrispondenti, normalizzati sul totale dei passi in cui la modifica è stata accettata.

2 Misure preliminari

2.1 Nota sulla termalizzazione

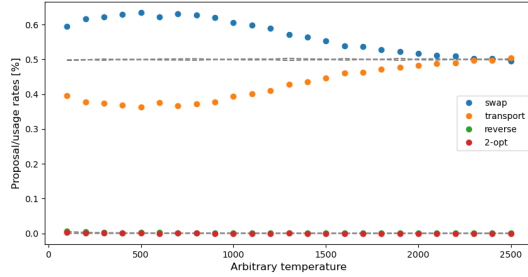
Come in un qualsiasi sistema soggetto a fluttuazioni statistiche dipendenti da influenze esterne, anche in questo caso sarebbe ragionevole aspettarsi che il sistema attraversi inizialmente un transiente di stabilizzazione prima di "termalizzare", vale a dire andare a regime. In realtà, si vede che la termalizzazione ad alte temperature avviene entro pochi passi, e cioè il sistema trova quasi subito la lunghezza minima che gli è consentita dall'esplorazione termica dell'*energy landscape*. A basse temperature, invece, non ha senso parlare di termalizzazione, perché per costruzione verranno accettate quasi sempre modifiche piuttosto vantaggiose e la lunghezza totale del percorso decrescerebbe costantemente, fino a saturare su una lunghezza che sarà generalmente quella di un minimo superficiale.

2.2 Ricerca della temperatura critica

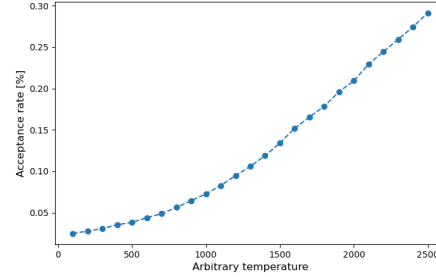
Per come sono stati costruiti gli operatori, è immediato dedurre che:

$$T \rightarrow \infty \implies \begin{cases} p(\text{swap}) \sim 0.5 \sim p(\text{transport}) \\ p(2\text{-opt}) \sim 0 \end{cases} \quad (2.2.1)$$

$$T \rightarrow 0 \implies \begin{cases} p(\text{swap}) \sim 0 \sim p(\text{transport}) \\ p(2\text{-opt}) \sim 1 \end{cases} \quad (2.2.2)$$

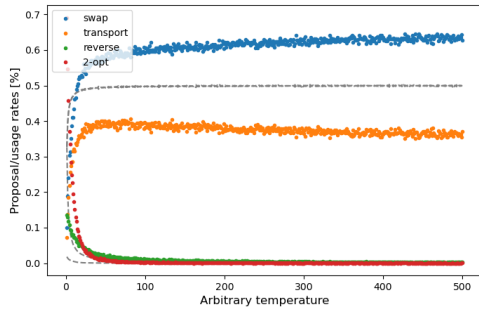


(a) Rates degli operatori proposti e usati.

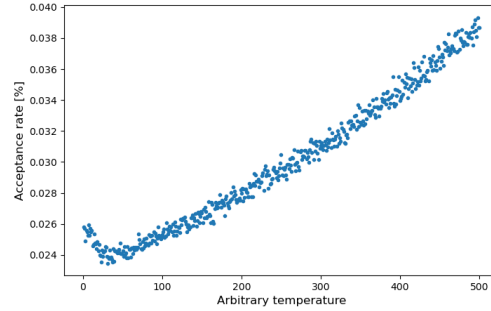


(b) Percentuale di accettazione.

Figura 2.2 – Scansione fine in temperatura. Le linee tratteggiate grigie rappresentano i rates di proposta dei vari operatori, che rimangono a 0 per *reverse* e *2-opt* e a 0.5 per *swap* e *transport*. I dati colorati sono invece i rates di utilizzo corrispondenti, normalizzati sul totale dei passi in cui la modifica è stata accettata.



(a) Rates degli operatori proposti e usati.



(b) Percentuale di accettazione.

Figura 2.3 – Scansione finissima in temperatura. Le linee tratteggiate grigie rappresentano i rates di proposta dei vari operatori, che rimangono a 0 per *reverse* e *2-opt* e a 0.5 per *swap* e *transport*. I dati colorati sono invece i rates di utilizzo corrispondenti, normalizzati sul totale dei passi in cui la modifica è stata accettata.

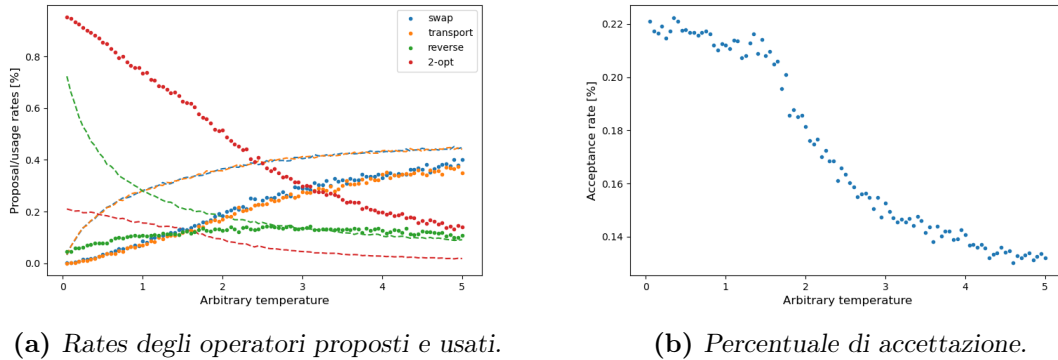


Figura 2.4 – Scansione iperfina in temperatura. Qui si distinguono bene sia i rates di proposta (linee tratteggiate) che quelli di utilizzo (marker circolari).

Non è invece altrettanto immediato cogliere attorno a che temperatura si distinguono i due regimi. In particolare, il limite per $T \rightarrow \infty$ verrà soddisfatto per $T > T_C^+$ e quello per $T \rightarrow 0$ in $T < T_C^-$. Non possiamo assumere che $T_C^+ = T_C^-$, in linea di principio, e infatti le prove in figura 1.3 sembrano suggerire due diversi *shift* nell'andamento del sistema in due diversi regimi di temperatura. Non solo: in assenza di indicazioni fisiche sulla temperatura, che peraltro è definita in maniera del tutto arbitraria, non è banale dedurre l'ordine di grandezza. In linea di massima, le temperature critiche dipenderanno anche dalla variazione tipica di lunghezza $\Delta\ell(\pi \rightarrow \pi')$ entro un dato sistema, che però a sua volta è una quantità non banale da stimare.

In generale, è opportuno considerare i dati presenti su TSPLIB e inserirsi in una media ragionevole. Reticoli quadrati di lato 10^4 e 100 città sembrano essere un buon compromesso. Viste le prove in figura 1.3, possiamo supporre che valutare temperature oltre l'ordine delle migliaia sia superficiale. Perciò, svolgiamo quattro scansioni in temperatura, sempre più raffinate.

1. **scan grossolano** (fig. 2.1) - fino a $T = 10\,000$, a passi di $\Delta T = 500$. Temperature nell'ordine delle migliaia soddisfano già il limite a infinito, portando gli operatori grossolani a prevalere del tutto sugli altri e a compensarsi a vicenda.
2. **scan fine** (fig. 2.2) - $T = 2500$, $\Delta T = 100$. Attorno a $T = 2500$ i protocolli grossolani si biforcano: sotto a questa temperatura osserviamo una prima distinzione "intelligente" nell'uso degli operatori grossolani. Possiamo affermare che $T_C^+ = 2500$.
3. **scan finissimo** (fig. 2.3) - $T = 500$, $\Delta T = 1$. L'unico cambiamento notevole è per temperature bassissime, dove osserviamo i rates cambiare sensibilmente. Nel frattempo, il tasso di accettazione è infimo, ma sembra risalire in prossimità dello zero.

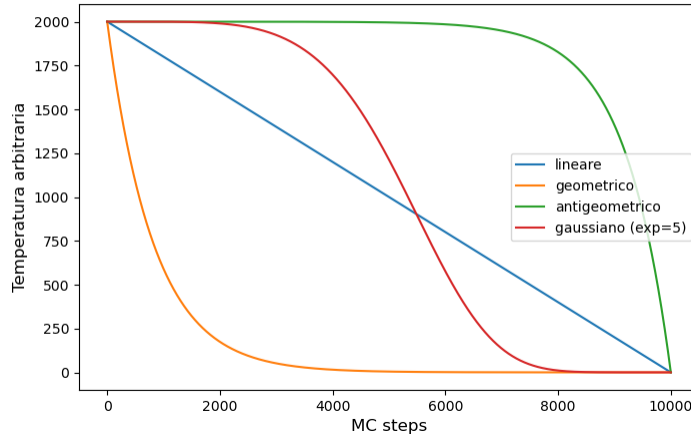


Figura 2.5 – Protocolli di raffreddamento proposti. Per esigenze di visualizzazione e per distinguere nettamente l'andamento, il protocollo gaussiano è qui presentato con un esponente anomalo 5.

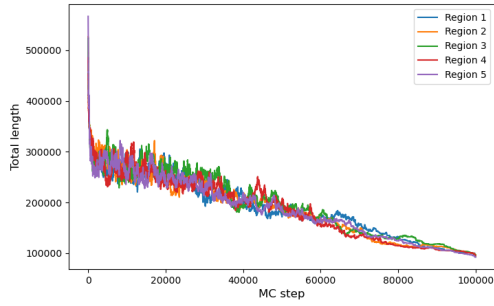
4. **scan iperfine** (fig. 2.4) - $T = 5$, $\Delta T = 0.05$. Non serve dimostrare che per $T = 1$ tutte le probabilità di proposta degli operatori valgono 0.25, ma il grafico non rispetta la deduzione. Questo avviene perché non sempre il percorso presenta intersezioni. Sotto a $T = 1$ l'operatore **2-opt** diventa sensibilmente più significativo, e gli altri vanno a decadere. Risale anche il tasso di accettazione: è chiaro che $T_C^- = 1$.

In sintesi, si trova che il raffreddamento del sistema dovrebbe partire da una temperatura non oltre $T = 2500$, oltre la quale non ci si può aspettare alcun miglioramento significativo. Il sistema inoltre deve trascorrere un tempo significativo (cioè un sufficiente numero di passi Monte Carlo) al di sotto di $T = 1$, per garantire che i protocolli finiti possano affondare il sistema nel minimo che è stato esplorato.

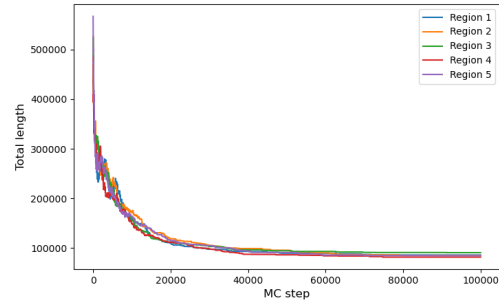
2.3 Protocolli di raffreddamento

Trovati gli ordini di grandezza delle temperature da considerare, serve ora scegliere l'andamento ottimale del raffreddamento. A livello euristico, questo può avvenire in quattro modi; vedasi figura 2.5 per una panoramica in merito. Il protocollo più semplice è quello lineare: per ogni passo, la variazione negativa di temperatura è costante. Altrimenti, il sistema può venire raffreddato rapidamente nei primi passi Monte Carlo e mantenuto freddo alla fine (protocollo "geometrico"¹); oppure, viceversa, può rimanere ad alte temperature fino a che

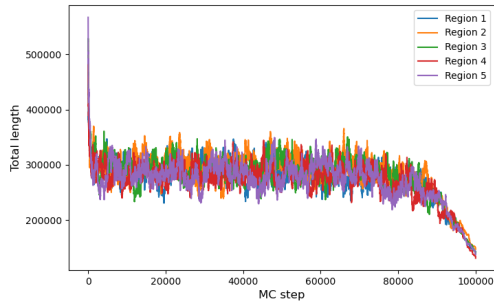
¹I nomi "geometrico" e "antigeometrico" derivano dalla funzione di libreria di NumPy `numpy.geomspace`, che ha la stessa funzione di `numpy.linspace` ma distanzia "geometricamente" i valori, cioè assicurandosi



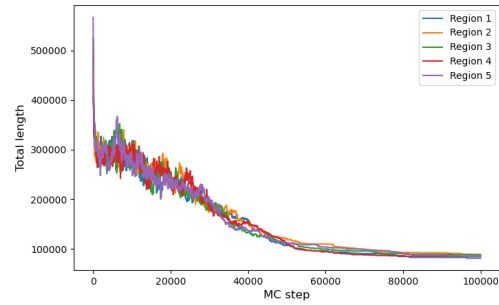
(a) Raffreddamento lineare.



(b) Raffreddamento "geometrico".



(c) Raffreddamento "antigeometrico".



(d) Raffreddamento gaussiano.

Figura 2.6 – Confronto pratico tra i vari protocolli di raffreddamento.

non viene rapidamente raffreddato in prossimità della fine (protocollo "antigeometrico"). Infine, secondo il protocollo "gaussiano", il sistema viene raffreddato poco all'inizio e alla fine, mentre la variazione di temperatura più marcata avviene nel mezzo della simulazione.

Per valutare quale protocollo porti i risultati migliori vengono generate cinque regioni con i parametri decisi sopra e vengono fatte evolvere per 100 000 passi Monte Carlo dalle stesse condizioni iniziali, con i quattro protocolli di raffreddamento. I risultati sono in figura 2.6. Ciascun protocollo parte da $T_0 = 2000$ e termina a $T_f = 0.01$ lungo 10 000 passi Monte Carlo. In particolare, il protocollo gaussiano segue in realtà una gaussiana generalizzata e non normalizzata:

$$G(x) = T_0 \exp \left\{ -\frac{1}{2} \left(\frac{x}{n/d} \right)^\alpha \right\} + T_f \quad (2.3.1)$$

Nello specifico, α è un esponente "anomalo" della distribuzione, che è proporzionale alla

che ogni numero sia un multiplo del precedente. Il protocollo antigeometrico non è che il simmetrico del protocollo geometrico rispetto alla retta tracciata dal protocollo lineare.

rapidità del raffreddamento (la pendenza del versante della gaussiana), e n/d è una varianza, anch'essa anomala, legata alla lontananza del versante della gaussiana dall'origine. Se n è il numero di passi Monte Carlo, d è proporzionale alla risoluzione della distribuzione.

I risultati nel grafico di fig. 2.6 sono di certo molto interessanti. Il protocollo anti-geometrico è insufficiente: il sistema non ha tempo di esplorare a sufficienza un minimo soddisfacente, perché rimane troppo tempo ad alte temperature. Gli altri tre protocolli raggiungono tutti risultati simili, toccando la soglia minima di $\ell = 10^5$, ma con andamenti diversi.

Il protocollo lineare è un ottimo candidato: la lunghezza sembra decrescere in maniera costante, anche se si può ragionevolmente supporre che l'andamento raggiunga un plateau oltre i 100 000 MC step. Raffreddando "geometricamente" il sistema si ottiene un risultato curioso: il sistema spende poco tempo ad alte temperature, quindi ha poco tempo per esplorare l'*energy landscape*, ma sembra comunque raggiungere basse lunghezze. Tuttavia, bisogna segnalare che il sistema satura subito il minimo e potrebbe bloccarsi in un plateau insoddisfacente.

Il raffreddamento gaussiano sembra essere il più promettente: il sistema raffredda poco all'inizio, esplorando capillarmente i minimi a disposizione, e una graduale discesa lo porta a stabilizzarsi a lunghezze molto basse. Questo protocollo può essere tarato con facilità variando α e d come definite nell'eq. (2.3.1). Inoltre, in letteratura un suggerimento comune è quello di lasciare il sistema ad alte temperature per un certo lasso di tempo prima di raffreddarlo.

Pertanto, si procede con il protocollo di raffreddamento gaussiano, del quale vanno trovati i parametri ottimali.

2.4 Ottimizzazione del protocollo gaussiano

Riprendendo l'eq. (2.3.1):

$$G(x) = T_0 \exp \left\{ -\frac{1}{2} \left(\frac{x}{n/d} \right)^\alpha \right\} + T_f \quad (2.4.1)$$

elaboriamo degli *educated guesses* in merito ai valori che α e d possono ragionevolmente assumere.

L'esponente $\alpha = 5$ è quello usato per il protocollo gaussiano in figura 2.6 e 2.5 e sembra essere già eccessivo e troppo ripido. Pertanto, scegliamo di variare $\alpha \in [2, 3, 4, 5]$.

Il fattore di scala d della varianza anomala non può essere più piccolo di 2, perché così l'andamento si avvicinerebbe a quello antigeometrico e risulterebbe massimamente inefficace. D'altra parte, è insensato che sia troppo grande, perché porterebbe all'andamento opposto, geometrico. Pertanto, un intervallo ragionevole potrebbe essere $d \in [3, 4, 5, 6]$.

In tabella 2.1 vengono presentati i risultati dei vari protocolli di raffreddamento, sotto forma degli intervalli di lunghezza entro cui il sistema oscilla alla fine della simulazione. I

parametri sono quelli della sezione precedente: 100 000 Monte Carlo steps, 5 regioni generate e fatte evolvere con seed differenti.

α	$d = 3$	$d = 4$	$d = 5$	$d = 6$
2	(8.0, 10.0)	(8.9, 9.0)	(8.0, 8.8)	(7.5, 9.0)
3	(8.5, 9.5)	(8.0, 9.5)	(8.0, 9.0)	(8.3, 8.9)
4	(7.8, 8.8)	(8.1, 9.0)	(8.3, 9.0)	(8.4, 9.0)
5	(8.3, 9.2)	(8.0, 9.2)	(8.3, 9.2)	(8.5, 9.1)

Tabella 2.1 – Intervalli di lunghezza entro cui il sistema si stabilizza se viene raffreddato con una gaussiana come in eq. (2.3.1) con α e d indicati. I valori sono da intendersi moltiplicati per 10^3 .

I risultati migliori si hanno per $(\alpha, d) = (2, 5), (4, 3)$. La colonna di destra va scartata, perché negli ultimi 10^4 MC step il tasso di accettazione degli operatori si aggira attorno allo 0.001% e risulta inefficiente. Per il motivo opposto, va scartata la prima colonna, dove i valori oscillano molto senza produrre risultati sensibilmente migliori. Quando α assume valori grandi, le stime sembrano peggiorare, probabilmente perché il raffreddamento è molto repentino.

In conclusione, i parametri migliori per la gaussiana sembrano essere $(\alpha, d) = (2, 5)$.

3 Calcolo e risultati

Le operazioni preliminari suggeriscono i valori da attribuire alle simulazioni per ottenere i risultati migliori. Riassumendo:

- non serve tenere conto di un tempo di termalizzazione;
- le temperature "critiche" sono $T_C^+ = 2500$ e $T_C^- = 1$, e pertanto è ragionevole che il sistema vada fatto raffreddare da $T = 2000$ a $T = 0.01$;
- il protocollo di raffreddamento ideale è quello gaussiano, con $(\alpha, d) = (2, 5)$.

Fissati questi valori, applichiamo le routines ai TSP già noti e caricati su TSPLIB. In particolare, ci riferiamo ai files già riportati in [2]. Per ciascun sistema, si prepara una simulazione di $2 \cdot 10^5$ passi MC da ripetere per 20 seed differenti. La lunghezza finale raggiunta viene mediata e l'errore corrisponderà alla deviazione standard. Dividiamo le simulazioni in tre batterie come segue, riportando le sigle ufficiali:

1. **problemi simmetrici semplici**² (TSP) - `gr21`, `fri26`, `dantzig42`
2. **problemi antisimmetrici semplici** (ATSP) - `ftv33`, `ftv38`, `ft53`
3. **problemi antisimmetrici complicati**³ (ATSP⁺) - `kro124p`, `ftv170`, `rbg323`, `rbg358`, `rbg403`, `rbg443`

File	Minimo	Risultati			
		AS	PMX	OX	CX2
<code>gr21</code>	2707	2707 ± 0	3127	3208	3145
<code>dantzig42</code>	699	937 ± 17	1425	1301	802

Tabella 3.1 – *Problemi TSP.*

Va tenuto presente che, per problemi di hardware dovuti al terminale utilizzato, qualsiasi *run* che superasse i $2 \cdot 10^5$ passi Monte Carlo bloccava il sistema, rendendo necessario un riavvio forzato. Pertanto, ci limiteremo a questa statistica, che comunque si rivelerà essere più che sufficiente.

Segnaliamo inoltre che l'analisi dei files corrispondenti a `gr21` e `rbg323` non è stata completata e non è presente nelle tabelle sottostanti. Il codice utilizzato, infatti, non sostiene in qualche modo questi calcoli: in particolare, il kernel Python utilizzato va sistematicamente

²Nel caso di `gr21`, il sistema raggiunge il minimo assoluto con qualsiasi seed! Invece, nel caso di `fri26`, il kernel va sistematicamente in crash.

³Anche nel caso di `rbg323`, il kernel va sistematicamente in crash.

File	Minimo	Risultati			
		AS	PMX	OX	CX2
ftv33	1286	1373 ± 41	2012	2098	2083
ftv38	1530	1590 ± 27	2578	2617	2560
ft53	6905	7536 ± 265	14949	14724	12243

Tabella 3.2 – *Problemi ATSP.*

File	Minimo	Risultati			
		AS	PMX	OX	CX2
kro124p	36 230	$42\,288 \pm 1323$	100 335	103 457	101 229
ftv170	2755	5515 ± 162	16 775	17 569	7019
rbg358	1164	2348 ± 71	5532	5830	5622
rbg403	2465	3553 ± 56	6536	6386	6455
rbg443	2720	3984 ± 56	6905	7121	6981

Tabella 3.3 – *Problemi ATSP⁺.*

in crash a prescindere dal terminale utilizzato, dalle versioni installate dei pacchetti, dai seed inseriti e dai parametri della simulazione.

Infine, nelle tabelle le sigle vogliono significare *Simulated Annealing* (SA), *Partially Mapped Crossover Operator* (PMX), *Order Crossover Operator* (OX) and *New Crossover Operator* (CX2).

4 Conclusione

In ultima analisi, il codice proposto applica con efficacia il metodo di *Simulated Annealing* a TSP di ogni genere, anche molto complicati, con siti da visitare nell'ordine delle centinaia. L'algoritmo resta molto flessibile e lascia spazio a ottimizzazioni più capillari, ad esempio tarando i parametri della gaussiana di raffreddamento su una specifica classe di problemi.

Il codice proposto raggiunge in maniera consistente minimi della lunghezza di percorrenza molto soddisfacenti, escludendo problematiche del kernel già menzionate e slegate dalla logica dell'algoritmo. Spesso, il minimo assoluto è entro poche deviazioni standard dal valore medio trovato.

In particolare, i risultati esposti sono marcatamente migliori di quelli trovati da algoritmi genetici. Questo tuttavia non rende il SA un metodo automaticamente preferibile: le generazioni nel caso di GA sono due ordini di grandezza inferiori in numero agli step MC qui richiesti. In [2], le generazioni sono 500 oppure 1000, mentre nella presente le simulazioni vanno su un numero di step nell'ordine delle decine di migliaia.

Riportiamo infine in figura 4.1 l'andamento dei minimi trovati rispetto al minimo assoluto, in funzione del numero di città.

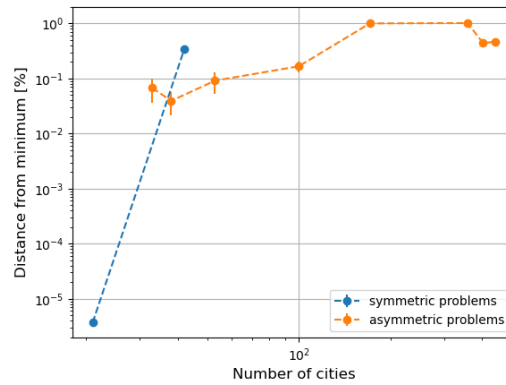


Figura 4.1 — Andamento della precisione in funzione del numero di città. La precisione viene fornita come distanza percentuale dal minimo ottimale, in valore assoluto.

Riferimenti bibliografici

- [1] H. Gould, J. Tobochnik e W. Christian. An Introduction to Computer Simulation Methods. 3^a ed. Addison-Wesley, 2007.
- [2] Abid Hussain et al. “Genetic Algorithm for Traveling Salesman Problem with Modified Cycle Crossover Operator”. In: Computational Intelligence and Neuroscience 2017 (2017), Article ID 7430125. DOI: 10.1155/2017/7430125.
- [3] William H. Press et al. Numerical Recipes in Fortran 77: The Art of Scientific Computing. Cambridge University Press, 1992.